

Trabalho 1 de Sistemas Operacionais

Lucas Emanuel de Oliveira Santos

Rafael Urbanek Laurentino

3 de junho de 2026

1 Introdução

Este relatório descreve detalhadamente a implementação de um sistema de sincronização de múltiplos processos utilizando Semáforos e Memória Compartilhada.

2 Execução do Programa

O programa pode ser compilado e executado de duas formas distintas.

2.1 Utilizando o Script `roda.sh`

A forma mais simples consiste em utilizar o script fornecido junto ao projeto. O script realiza automaticamente a compilação do programa através do `make`, executa os testes para diferentes quantidades de processos e, ao final, executa o `make clean`.

Para utilizar o script:

```
chmod +x roda.sh
./roda.sh
```

Durante a execução serão gerados arquivos de log contendo a saída dos testes para cada configuração utilizada, permitindo a análise posterior dos resultados.

2.2 Compilação e Execução Manual

Também é possível compilar e executar o programa manualmente.

Primeiramente, realize a compilação através do comando:

```
make
```

Após a compilação, o programa pode ser executado informando o número de processos desejado como argumento de linha de comando:

```
./trab <numero_de_processos>
```

Por exemplo, para executar o programa com 10 processos:

```
./trab 10
```

Os arquivos gerados pela compilação podem ser removidos utilizando:

```
make clean
```

3 Detalhes da Implementação

3.1 Implementação da Barreira

A barreira funciona em duas fases distintas:

- **Fase 1 (Chegada/Entrada):** Cada processo incrementa o contador global `count` de forma protegida. Os primeiros $n - 1$ processos ficam bloqueados na primeira catraca (`turnstile1`). Quando o n -ésimo (último) processo chega, ele fecha a segunda catraca e abre a primeira, disparando um efeito cascata que libera todos os processos.
- **Fase 2 (Saída):** À medida que os processos saem, eles decrementam o mesmo contador. O último processo a sair fecha a primeira catraca (preparando-a para um uso futuro) e abre a segunda catraca (`turnstile2`).

O uso de uma única catraca pode gerar uma vulnerabilidade conhecida como “Condição de Corrida de Barreira”. Se usássemos apenas uma catraca poderia acontecer o seguinte cenário:

1. O último processo abre a catraca.
2. Um processo muito rápido passa pela catraca, executa o código seguinte, dá a volta no loop do programa e reentra na barreira antes que os processos mais lentos tivessem saído da primeira fase.
3. Isso corromperia o contador `count` e prenderia processos na barreira indevidamente.

A introdução da segunda catraca (`turnstile2`) resolve isso criando um sistema de “fechamento duplo”:

- A primeira catraca controla a **ENTRADA** e garante que ninguém passa enquanto todos não chegarem.
- A segunda catraca controla a **SAÍDA** e garante que **NENHUM** processo reinicie a barreira ou avance para um novo ciclo até que todos os processos tenham saído da primeira fase.

Isso torna a barreira segura e reutilizável.

3.2 Implementação da Fila (FIFO)

A estrutura `FifoQT` implementa o acesso exclusivo a um recurso, mas com uma garantia de ordem estrita: quem chega primeiro, é atendido primeiro. A fila mantém um array fixo de semáforos (`slots`), um ponteiro para o próximo processo a ser atendido (`head`), um ponteiro para a próxima vaga livre (`tail`) e uma flag de controle (`em_uso`).

Quando um processo solicita o recurso, ele verifica se está livre e se a fila está vazia. Se estiver, ele pega o recurso imediatamente. Caso contrário, ele retira um “ticket” baseado na posição atual de `tail`, incrementa o `tail` e dá um `sem_wait` no seu semáforo exclusivo (`slots[meu_ticket]`). Isso garante que ele ficará dormindo exatamente na sua ordem de chegada.

Ao liberar o recurso, o processo verifica se `head < tail`, o que indica que há processos esperando. Em caso positivo, ele incrementa o `head` e dá um `sem_post` no semáforo do próximo processo da fila (`slots[proximo]`), acordando-o de forma ordenada. Se não houver ninguém, apenas zera `em_uso`.

3.3 Lógica de Processos Pai e Filhos

O programa foi desenhado como um modelo de concorrência onde o processo Pai também participa ativamente da carga de trabalho. O Pai executa um laço `for` de 1 até $n - 1$ chamando a função `fork()`. O processo filho gerado recebe um ID lógico (`nProc`), executa a função `rotina_processo` e encerra com `exit(0)`. O Pai salva o PID de cada filho em um array dinâmico (`pids_filhos`) para controle.

Após criar os filhos, o processo Pai não fica ocioso. Ele assume a identidade lógica `nProc = 0` e chama a mesma função `rotina_processo`, integrando-se à barreira e ao loop da FIFO.

Quando o Pai encerra sua `rotina_processo`, ele entra em um loop utilizando a chamada de sistema `wait()`. Esse loop captura o término de cada um dos processos filhos, identifica qual era o número lógico do filho através do mapeamento do array de PIDs e imprime uma mensagem de encerramento.

3.4 Uso da Função `mmap()`

A memória compartilhada foi implementada através da função `mmap()`, utilizando as flags `MAP_SHARED` e `MAP_ANONYMOUS`. Esta abordagem foi escolhida por múltiplos motivos estruturais.

Como a memória é alocada antes da criação dos filhos, todos eles herdam automaticamente o mesmo ponteiro válido para as estruturas, eliminando a necessidade de chamadas adicionais para acoplar a memória. A flag `MAP_SHARED` garante que qualquer alteração feita nas estruturas da barreira ou da FIFO por um processo seja imediatamente visível

para todos os outros, permitindo a sincronização real entre os processos.

Por sua vez, a flag `MAP_ANONYMOUS` aloca o espaço diretamente na RAM virtual, sem precisar criar, gerenciar ou apagar arquivos temporários no sistema (como exige o método `shm_open`).